Vibe Coding: from an HTML file to an Android APK

For non-coders. You describe the app in your own language. A Claude Code cloud session, embedded with a GitHub repository, writes the files. GitHub builds the APK.

Vibe coding means you judge the running app, and the model writes the program. Andrej Karpathy gave the practice that name on 2 February 2025. This manual is the version you can ship: every change is a GitHub commit, every installable file is built by a recorded run, and you can say, in plain language, what the app is allowed to do.

You stay in a browser, and you install the result on an Android phone. The product that does the writing is **Claude Code in the cloud**, at `claude.ai/code`. The ordinary Claude chat page can draft HTML. It does not clone your repository, push a branch, or leave you a build you can re-run. Use the cloud session.

1. The path

1. You create an empty GitHub repository.
2. You open a cloud session with that repository already selected. That selection is the embedding.
3. You describe the screen in the language you speak. Claude writes `www/index.html` and the files next to it.
4. In the same session, Claude wraps those files with Capacitor 8. Capacitor makes an Android app whose only screen is your HTML.
5. Claude adds a GitHub Actions workflow. GitHub's build machine has the Android SDK. The session's machine does not.
6. You download `app-debug.apk` and install it.
7. Each later change is a new message, a new branch, and a new APK.
8. When the app is worth keeping, you sign it with one key you store yourself. A version tag produces `app-release.apk` (install on a phone) and `app-release.aab` (upload to Google Play).

Stage	You send	You receive
HTML	What the screen does	`www/index.html`
Wrap	The Capacitor instructions in section 6	`android/` and `capacitor.config.json`
Debug	The workflow file in section 6	`app-debug.apk`
Update	One change, in your language	A new debug APK
Release	A GitHub Release tagged `v1.0.0`	Signed APK and AAB

The worked example is a one-screen errand list. Replace the name, the app id, and the description before you send the first message. Use the same three values in every later message.

Placeholder	Example value	Rule
App name	`Errand List`	The name under the icon. You may change it later.
App id	`com.example.errandlist`	Lowercase. At least two parts separated by dots. Each part starts with a letter. Then only letters, digits, and underscores. It is permanent. On your own phone it must be unique among your apps. On Google Play it must be unique worldwide. Use your own id, for example `com.maria.errands`.
Description	The "App:" paragraph in message 1	Write this in your language. It is the whole product.

2. Accounts and the GitHub connection 2
3. Embed the repository in a session 3
4. How to write a message 4

5. Your own language 4
6. Session 1: HTML, Android project, build 4
7. Install the debug APK 8
8. The next change 9
9. CLAUDE.md, the standing rules 9
10. The signing key 9
11. Signed APK and AAB 10
12. Google Play 10
13. Building on the session machine 11
14. Failures and the reply that fixes them 12
15. What this APK can do 12
16. Before anyone else installs it 13
17. Sources 13

2. Accounts and the GitHub connection

You need a GitHub account, a Claude subscription that includes cloud sessions, a browser, and an Android phone. Cloud sessions are included on Pro, Max, and Team plans, and on Enterprise seats that include Claude Code. The session counts against that plan's usage. There is no separate charge for the virtual machine. Signing in with an API key does not start a cloud session.

Create the repository

1. On GitHub, choose **New repository**.
2. Name it with letters, numbers, and hyphens, for example `errand-list`.
3. Choose Private or Public. A private repository works after the Claude GitHub App is installed on it.
4. Check **Add a README file**. The session needs a branch to clone. Leave the license and the ignore template empty.
5. Create the repository. The address is `https://github.com/OWNER/REPO`. You will type `OWNER/REPO`.

Connect Claude Code to GitHub

1. Open `https://claude.ai/code`.
2. Choose **Sign in with GitHub** and approve the request. GitHub sends you back to `claude.ai/code`.
3. For a private repository, install the **Claude GitHub App** on the account or organization that owns it, and include this repository. An organization owner may have to approve the install. The install also turns on Auto-fix for pull requests in that repository.
4. On Pro and Max, onboarding creates an environment named **Default** with Trusted network access. On Team and Enterprise, leave the prefilled name and network access as they are and choose **Create & finish**. If the page says GitHub access is required, an Owner first turns on the GitHub connector at Organization settings → Connectors, then you reload.

Default is the right environment for this manual. Trusted access reaches GitHub, npm, Maven, and Gradle. It does not include `dl.google.com`, which is where Android SDK packages are downloaded. That is why section 6 builds the APK on GitHub Actions, where the SDK is already installed. Section 13 is the optional path that adds `dl.google.com` and builds inside the session.

> Organizations with Zero Data Retention, the HIPAA configuration, or customer-managed encryption keys cannot start these cloud sessions. If the page says cloud sessions are unavailable, that setting is the reason.

The same connection from a terminal

Use this only if you already use the `gh` command. In a shell: `gh auth login`, then `gh auth refresh -s workflow`. Open the Claude Code CLI, run `/login` with the same claude.ai account, then run `/web-setup`

and confirm. The `workflow` scope is what allows a later push of `.github/workflows`. On Team and Enterprise, `/web-setup` appears after an Owner turns on Quick setup. Disconnecting GitHub at `claude.ai/customize/connectors` removes the token from cloud sessions. Your local `gh` login stays until you revoke it on GitHub.

3. Embed the repository in a session

A cloud session runs on an Anthropic-managed virtual machine: Ubuntu 24.04, x86_64, about 4 vCPUs, 16 GB of RAM, and 30 GB of disk. It clones the repository you selected. You do not get a shell. You type what you want, and Claude runs the commands. Closing the browser tab leaves the session running. After a few minutes with no messages, the machine pauses. Open the session again from the sidebar. If the page says the environment expired, the conversation is restored on a new machine; a command that was still running is not restored.

From the browser or the phone

1. Open `claude.ai/code`, or the **Code** tab in the Claude mobile app.
2. Under the message box, open the repository selector and choose `OWNER/REPO`. Add no second repository. A session with one repository loads `CLAUDE.md` and the repo's Claude settings. A session with several repositories does not load hooks from `.claude/settings.json`.
3. Leave the branch on the default branch for the first session. Claude will create `html-app`. For a later session that must continue unmerged work, change the branch in the control next to the repository.
4. Set the permission mode. **Plan**: Claude proposes the approach and waits. **Accept edits**: Claude edits files and pushes. **Auto**: a classifier reviews actions when your plan allows it. Cloud sessions have no Manual mode and no Bypass mode.
5. Paste the message. Press Enter.

Start message 1 in **Plan**. When the plan matches the message, reply `Approved. Build it and push the branch.` Switch the mode to **Accept edits** before message 2.

Embed with a URL

This URL opens a session form with the repository and a short instruction already filled in. Replace `OWNER/REPO`. Encode spaces as `%20`. Keep the URL short. Put long instructions in the text box, or, after section 9, in `CLAUDE.md`.

SESSION URL

```
https://claude.ai/code?repositories=OWNER/REPO&environment=Default&prompt=Follow%20CLAUDE.md
```

Parameter	Meaning
`repositories`	Comma-separated `owner/repo` list. `repo` is an alias. This manual uses one.
`prompt`	Text placed in the box. `q` is an alias.
`prompt_url`	A URL that returns the prompt text. Ignored when `prompt` is also set. The URL must allow cross-origin reads. A raw GitHub URL usually does not.
`environment`	Environment name or id. Use `Default` until section 13.

From the Claude Code CLI

Push first. `claude --cloud` clones the GitHub branch, not uncommitted files on your laptop.

START, THEN SEND A FOLLOW-UP

```
claude --cloud "Follow CLAUDE.md. Add a delete-all button."
claude -p "Make the Add button full width." --cloud SESSION_ID
```

The session id is in the `claude.ai/code` link. To continue on your laptop, run `claude --teleport SESSION_ID` from a clean checkout of the same repository and the same claude.ai account. The branch must already be pushed.

Each task gets its own session and branch, so several tasks can run at the same time. For this app, finish session 1 and merge it before you start a second task. Two sessions editing the same HTML will produce two branches you would have to combine.

4. How to write a message

1. One change per message.
2. Name the visible result: what a person sees, what a tap does, what is still there after the app closes.
3. Name the file when you know it.
4. State what must stay: the app id, the signing setup, and the screens you did not mention.
5. When something fails, paste the error unchanged. Add one sentence: what you expected on the phone.
6. End with: commit, push the branch, leave `main` unchanged.
7. Read the diff by behavior. You are checking that the words you asked for are present, that `appId` is yours, and that no `.jks` file appears.

The diff view shows a count such as `+42 -18`. Open it. Select a line, type the correction, and press Enter. The comment waits. Your next message sends it with the rest of the instruction, including the file and line. When the diff is right, choose **Create PR** at the top of the diff view and open a full pull request. Use that button. Inside this session, `gh pr create` calls GitHub GraphQL, and the session proxy answers `GitHub GraphQL is not available from Claude Code sessions`.

5. Your own language

Write every message in the language you already speak. Claude reads that message. The app's on-screen language is a separate instruction. Say both.

You want	Add this sentence
English screen	The visible language is English. Lay the screen out left to right.
Another left-to-right language	The visible language is Spanish. Lay the screen out left to right. La pantalla está en español. El texto va de izquierda a derecha.
A right-to-left language	The visible language is Urdu. Lay the screen out right to left. Store the font file under `www/fonts/` and use it in the CSS.

ایپ کی زبان اردو ہو۔ سکرین دائیں سے بائیں ہو۔

Keep these tokens in ASCII even when the rest of the message is not English: file names, `www/`, `appId`, `versionCode`, `versionName`, the app id, and `.github/workflows/android-apk.yml`.

A font that is linked from the internet will load only when the phone is online. For a right-to-left app that must work offline, the message must say: put the font file in the repository and refer to it with a relative path.

6. Session 1: HTML, Android project, build

Use one session for the three messages below. Wait until Claude says the branch is pushed before you send the next message. Replace the app name, the app id, and the "App:" paragraph in all three before you send the first.

Message 1 creates the HTML. Message 2 wraps it. Wrapping needs Node.js and the Capacitor packages. The cloud image already has Node.js 22, npm, and OpenJDK 21. `npx cap add android` copies a project

template. It does not need the Android SDK, and it was run that way while this manual was written. `./gradlew assembleDebug` does need the SDK. Message 2 tells Claude not to run Gradle. Message 3 makes GitHub run it.

MESSAGE 1 — PASTE, AFTER REPLACING THE THREE PLACEHOLDERS

```
Work in this repository on a new branch named html-app.

Build a phone app as HTML, CSS, and JavaScript. I will judge it on a phone. I will not edit the files.

App: a private errand list. One screen. A text box, an Add button, and the list. Tapping a row crosses it
out. Tapping a crossed-out row deletes it. Save the list in localStorage. Show the version 1.0.0 at the
bottom.

Create:
www/index.html
www/styles.css
www/app.js

Constraints:
The visible language is English. Lay the screen out left to right.
index.html contains a head element and a viewport meta of width=device-width, initial-scale=1.
CSS and JavaScript live only in www/styles.css and www/app.js.
The app works with the phone offline. No CDN and no remote font.
Body text is at least 18px. Buttons are at least 48px tall.
A 360px-wide screen does not scroll sideways.

Commit and push the branch. Leave main unchanged.
```

In the diff, confirm `www/index.html` exists and the screen text from your “App:” paragraph is in the files. Then send message 2.

MESSAGE 2 — WRAP WITH CAPACITOR 8

```
On this same branch, wrap www/ with Capacitor 8. Do not run ./gradlew. This session has no Android SDK.

Run, in this order:
npm init -y
npm install @capacitor/core@8 @capacitor/android@8
npm install -D @capacitor/cli@8
npx cap init "Errand List" com.example.errandlist --web-dir www
npx cap add android
npx cap sync android

In android/app/build.gradle set versionCode to 1 and versionName to "1.0.0".
In android/app/src/main/res/values/strings.xml set app_name and title_activity_main to Errand List.

Uncomment the *.jks and *.keystore lines in android/.gitignore.
Add a root .gitignore with these two lines:
node_modules/
*.jks
*.keystore

capacitor.config.json must contain:
appId com.example.errandlist
appName Errand List
webDir www
It must not contain server.url.

Commit package.json, package-lock.json, capacitor.config.json, .gitignore, and the android directory. Push
the branch.
```

On 20 October 2026, `@8` installed Capacitor 8.5.3. The lockfile keeps that exact version. The generated Android template uses `minSdkVersion` 24, `compileSdkVersion` 36, `targetSdkVersion` 36, Android Gradle Plugin 8.13.0, and Gradle 8.14.3. Phones below Android 7.0 cannot install this app.

`android/.gitignore` ignores `*.apk` and ignores `app/src/main/assets/public`. The HTML you edit stays in `www/`. The workflow copies it into the Android project at build time with `npx cap sync android`. A build that skips that copy ships the last synced page, or an empty shell.

MESSAGE 3 — WORKFLOW, SIGNING BLOCK, AND STANDING RULES

```
On this same branch, create .github/workflows/android-apk.yml with the workflow I paste under this paragraph, unchanged.
In android/app/build.gradle, replace the existing buildTypes block with the signingConfigs block and the buildTypes block I paste after the workflow.
Create CLAUDE.md with the standing rules I paste after that.
Do not change appId. Do not commit a .jks file. Push the branch.
```

Paste the three blocks below in the same message, after that paragraph.

FILE: .GITHUB/WORKFLOWS/ANDROID-APK.YML

```
name: Android APK

on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-24.04
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"

      - name: Copy HTML into the Android project
        run: |
          npm ci
          npx cap sync android
          chmod +x android/gradlew

      - name: Install Android SDK 36
        run: |
          SDK="${ANDROID_SDK_ROOT:-$ANDROID_HOME}"
          SM=$(find "$SDK/cmdline-tools" -name sdkmanager \
            -type f | sort | tail -n 1)
          yes | "$SM" --sdk_root="$SDK" --licenses \
            >/dev/null || true
          "$SM" --sdk_root="$SDK" \
            "platforms;android-36" "build-tools;36.0.0"

      - name: Debug APK
        if: ${{ !startsWith(github.ref, 'refs/tags/v') }}
        working-directory: android
        run: ./gradlew assembleDebug --no-daemon

      - name: Upload debug APK
        if: ${{ !startsWith(github.ref, 'refs/tags/v') }}
        uses: actions/upload-artifact@v4
        with:
          name: app-debug
          path: android/app/build/outputs/apk/debug/app-debug.apk
          if-no-files-found: error
          retention-days: 30

      - name: Decode the release keystore
        if: ${{ startsWith(github.ref, 'refs/tags/v') }}
```

```
        env:
          ANDROID_KEYSTORE_BASE64: ${{ secrets.ANDROID_KEYSTORE_BASE64 }}
        run: |
          if [ -z "$ANDROID_KEYSTORE_BASE64" ]; then
            echo "Missing secret ANDROID_KEYSTORE_BASE64"
            exit 1
          fi
          echo "$ANDROID_KEYSTORE_BASE64" \
            | base64 -d > "$RUNNER_TEMP/upload.jks"

      - name: Release APK and AAB
        if: ${{ startsWith(github.ref, 'refs/tags/v') }}
        working-directory: android
        env:
          ANDROID_KEYSTORE_FILE: ${{ runner.temp }}/upload.jks
          ANDROID_KEYSTORE_PASSWORD: ${{ secrets.ANDROID_KEYSTORE_PASSWORD }}
          ANDROID_KEY_ALIAS: ${{ secrets.ANDROID_KEY_ALIAS }}
          ANDROID_KEY_PASSWORD: ${{ secrets.ANDROID_KEY_PASSWORD }}
        run: ./gradlew assembleRelease bundleRelease --no-daemon

      - name: Verify the release signature
        if: ${{ startsWith(github.ref, 'refs/tags/v') }}
        run: |
          SDK="${ANDROID_SDK_ROOT:-$ANDROID_HOME}"
          APKSIGNER=$(find "$SDK/build-tools" -name apksigner \
            -type f | sort | tail -n 1)
          "$APKSIGNER" verify --print-certs \
            android/app/build/outputs/apk/release/app-release.apk

      - name: Attach files to the GitHub Release
        if: ${{ startsWith(github.ref, 'refs/tags/v') }}
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          gh release upload "$GITHUB_REF_NAME" \
            android/app/build/outputs/apk/release/app-release.apk \
            android/app/build/outputs/bundle/release/app-release.aab \
            --clobber

      - name: Upload release files
        if: ${{ startsWith(github.ref, 'refs/tags/v') }}
        uses: actions/upload-artifact@v4
        with:
          name: app-release
          path: |
            android/app/build/outputs/apk/release/app-release.apk
            android/app/build/outputs/bundle/release/app-release.aab
          if-no-files-found: error
          retention-days: 30
```

The runner is pinned to `ubuntu-24.04` because the `ubuntu-latest` label moves to Ubuntu 26.04 in November 2026. Both images can build this project. The pin keeps the run on the image this file was written for. The SDK step installs platform 36 and build-tools 36.0.0, which match Capacitor 8.

REPLACEMENT INSIDE ANDROID/APP/BUILD.GRADLE

```
    signingConfigs {
        release {
            def keystorePath = System.getenv("ANDROID_KEYSTORE_FILE")
            if (keystorePath) {
                storeFile file(keystorePath)
                storePassword System.getenv("ANDROID_KEYSTORE_PASSWORD")
                keyAlias System.getenv("ANDROID_KEY_ALIAS")
                keyPassword System.getenv("ANDROID_KEY_PASSWORD")
            }
        }
    }
    buildTypes {
        release {
            if (System.getenv("ANDROID_KEYSTORE_FILE")) {
                signingConfig signingConfigs.release
```

```
            }
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
```

`signingConfigs` must sit above `buildTypes`, still inside the existing `android { }` block. A debug build leaves `ANDROID_KEYSTORE_FILE` empty, so Gradle uses its debug signature. A tag build sets the variable, so Gradle signs `app-release.apk` and `app-release.aab` with your key. `minifyEnabled false` leaves the WebView app uncompressed by R8. Turn that on only when you have a failing release that needs it, and expect a separate round of keep-rules.

FILE: CLAUDE.MD

```
# Standing rules

This is an HTML app packaged with Capacitor 8. The user writes messages, not code.

- Edit the UI only under www/. Then run npx cap sync android.
- webDir is www. Never add server.url.
- appId is com.example.errandlist. Never change it.
- Launcher name is app_name and title_activity_main in
  android/app/src/main/res/values/strings.xml.
- Do not edit android/app/src/main/assets/public by hand.
- Do not commit node_modules, build outputs, local.properties, or keystore files.
- Do not upgrade Capacitor unless the user asks.
- The on-screen version, versionName, and the Git tag must match.
  When the user asks for a release, read versionCode and add 1.
- Visible language and direction are the ones in the latest user message.
  Default: English, left to right.
- The page origin inside the APK is https://localhost.
- Push a branch. Do not push a tag. Do not merge.
- Do not run gh pr or gh issue.
- End every task with a plain-language list of Android permissions
  and every network host the app contacts.
```

Replace `com.example.errandlist` in `CLAUDE.md` with your app id before you send the message.

After the push, open the diff. Check the three files, the app id, and the absence of any `.jks` file. Choose **Create PR**. If Claude reports that GitHub rejected the workflow file, do section 14 before you look for an APK.

7. Install the debug APK

1. Open the pull request on GitHub. Wait until the **Android APK** check is green. The first run downloads Gradle and the Android libraries. A yellow spinner is a running build. A red X is a failed build; go to section 14.
2. Open the run, scroll to **Artifacts**, and download `app-debug`. You must be logged in. On a private repository, that download is part of your access.
3. Unzip the download. Install the file whose name ends in `.apk`. It is `app-debug.apk`, sometimes inside folders.
4. On the phone, open the APK. If Android blocks it, open Settings, search **Install unknown apps**, and allow the app you used to open the file (Chrome or Files).
5. Open the installed app. The screen is the one you described. The bottom shows `1.0.0`.

> Each debug build on GitHub Actions creates a new debug signature. The next debug APK will not update this one. Uninstall the app, then install the new APK. Uninstalling deletes the saved list. Section 10 is how you get a signature that stays, so later installs keep the data. Play Protect may warn on a debug APK. Choose install anyway only when the file came from this repository’s Actions run.

Use every control. Add an item, close the app, open it, and confirm the item is still there. That is the test of `localStorage`. Then, on GitHub, merge the pull request. The next session starts from `main`. Until you merge, point the next session at the branch `html-app`.

8. The next change

Open a new cloud session on `OWNER/REPO`, mode **Accept edits**. Send:

MESSAGE — ONE CHANGE

```
Follow CLAUDE.md.
Change: add a button that deletes every errand after a second tap to confirm.
Read the current versionCode in android/app/build.gradle and add 1.
Set versionName and the on-screen version to 1.1.0.
Run npx cap sync android.
Push a new branch. Leave main unchanged. Do not change appId.
```

Create the pull request, wait for the green check, uninstall the old debug app, and install the new APK. The version at the bottom is `1.1.0`.

To correct a line without a new essay, select that line in the diff, write the correction, and send the next message. To repair a red check, paste the log as in section 14, or turn on **Auto-fix** in the session's CI status bar. Auto-fix requires the Claude GitHub App on the repository. It answers CI failures and review comments. It does not see a merge conflict when `main` moves. If that happens, send `Rebase this branch onto main and push.`

Use **Auto** mode after one APK has installed and `CLAUDE.md` is on `main`.

9. CLAUDE.md, the standing rules

Message 3 already committed `CLAUDE.md` at the root of the repository. Every later cloud session on this one repository loads it. The copy of instructions on your laptop, in `~/.claude/CLAUDE.md`, is not loaded. Commit anything the next session must know.

The session also loads, for a single-repository session, `.claude/settings.json`, `.claude/rules/`, `.claude/skills/`, and a project-scoped `.mcp.json`. It does not load plugins that are enabled only on your laptop. You do not need any of those files for this app.

When a session's diff is wrong in the same way twice, add one line to `CLAUDE.md` that states the rule. That is the professional habit: the correction becomes a standing rule, and the next session starts with it.

10. The signing key

Android installs an update only when the new file is signed with the same key as the installed app and `versionCode` is higher. The debug APK from Actions uses a new debug key every run. A key you create once, and store, is the key for every later sideload update.

Create it on a computer you control. Java's `keytool` is enough. OpenJDK 21 is what this project builds with.

CREATE UPLOAD.JKS — RUN OUTSIDE THE REPOSITORY FOLDER

```
keytool -genkeypair -v \
  -keystore upload.jks \
  -storetype JKS \
  -alias upload \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000
```

Type a long password. When `keytool` asks for the key password, press Enter so the key uses the same password. Fill in the name it asks for. Validity of 10000 days from October 2026 ends in 2054, which is after the 22 October 2033 date Android requires of a release certificate.

Copy `upload.jks` to two places that are not the git repository: a password manager's file storage, and a second disk. A lost key means every sideload user must uninstall, which deletes the saved list. A new app id produces a second app, not an update.

ONE-LINE BASE64

```
base64 -w 0 upload.jks
base64 -i upload.jks | tr -d '\n'
```

The first command is Linux. The second is macOS. The output is one line.

On GitHub: repository → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Create these four. The names must match. Paste the base64 with no quotes and no spaces.

Secret	Value
`ANDROID_KEYSTORE_BASE64`	The one-line base64 of `upload.jks`
`ANDROID_KEYSTORE_PASSWORD`	The password you typed
`ANDROID_KEY_ALIAS`	`upload`
`ANDROID_KEY_PASSWORD`	The same password, if you pressed Enter for the key password

Add the secrets after the workflow from section 6 is merged to `main`. The debug job never reads them. The tag job reads them.

> Phone-only fallback, for a personal app: in a cloud session, ask Claude to run the `keytool` command above with `-storepass` and `-keypass`, print the base64, delete `upload.jks`, and confirm `git status` does not list it. Copy the base64 into the secret, then delete the session. Deleting a session permanently removes that session's data. The transcript contained the key until then. For an app other people depend on, create the key on a computer you control.

11. Signed APK and AAB

The cloud session proxy accepts branch pushes and rejects tag pushes. You create the tag in the browser. The tag starts the release half of the workflow.

1. Confirm `main` contains the version you want. The on-screen text, `versionName`, and the tag must be the same number. `versionCode` must be an integer higher than any build you already installed or uploaded.
2. On the repository, open **Releases** → **Draft a new release**.
3. In the tag box, type `v1.0.0` and choose **Create new tag** on `main`.
4. Title it `v1.0.0`. Publish the release.
5. Wait for the **Android APK** workflow on that tag. Refresh the release page. Download `app-release.apk` and `app-release.aab`. If the release page has no files and the workflow is green, download the `app-release` artifact from the run. Artifacts are deleted after 30 days. Keep the APK and AAB you ship.
6. The verify step prints the certificate. The alias in that printout is `upload`.

Uninstall the debug app first. Its signature is different, and Android will refuse the release APK until the debug copy is gone. Install `app-release.apk`. Add a list item. Then ship `v1.0.1` with `versionCode` increased by 1, and install that APK over the top. The list item must still be there. That test is the proof the key and the version rules are right.

Keep the APK for sideload. Keep the AAB for Play. Play will not accept the APK as a new app's package.

12. Google Play

New apps enroll in Play App Signing. You upload an AAB signed with your upload key. Google signs the APKs that Play delivers, with a key Google holds. If you lose the upload key, Play can reset it. People who

installed from Play keep receiving updates. People who installed your sideload APK do not: the sideload file is signed with the upload key, and the Play file is signed with Google's key. Those two installs do not replace each other. Uninstall one before installing the other.

Leave Google as the holder of the app signing key. That is the default when you create the app.

1. Open `https://play.google.com/console` and create the developer account. Pay the one-time fee shown on that page.
2. Create the app. The package name must equal `appId` character for character.
3. Complete the dashboard tasks the console lists: store listing, the screenshots in the sizes the form names, app access, ads declaration, content rating, target audience, and data safety.
4. For this errand list, data safety is: the list stays on the device in WebView storage, and the app sends no account and no list contents anywhere. Answer the form to match the code. If the form asks for a privacy-policy URL, ask Claude to write `docs/privacy.html` that says the same thing, merge it, then in the repository open Settings → Pages → branch `main` → folder `/docs`. The page is `https://OWNER.github.io/REPO/privacy.html`.
5. Open Testing → Internal testing → Create release. Upload `app-release.aab`. Add tester emails. Send the opt-in link. Testers install from Play.
6. Every later upload needs a higher `versionCode`. Produce it with a new tag, as in section 11.

Move from internal testing to production when the dashboard has no remaining required task and the testers have installed the build you intend to ship.

13. Building on the session machine

Use this when you want the APK before a pull request exists. The reliable build remains section 6.

At `claude.ai/code`, create an environment named `android-build`. Set network access to **Custom**, include the default domain list, and add one domain: `dl.google.com`. Set two variables:

```
BASH_DEFAULT_TIMEOUT_MS=600000
BASH_MAX_TIMEOUT_MS=1800000
```

Those are 10 minutes and 30 minutes. The defaults are shorter: Claude waits 2 minutes for a foreground command and can ask for up to 10, then the command can continue in the background for up to 30 minutes. A first Gradle build often needs the longer window.

Do not put the SDK install in the environment's setup script. A setup script is cached only when it finishes in about five minutes. The SDK download is larger than that budget. A script that exceeds it is not cached, and a script that exits non-zero blocks the session from starting.

MESSAGE — BUILD HERE, THEN PUSH THE APK

```
Install the Android SDK into /opt/android-sdk:
command-line tools, platforms;android-36, and build-tools;36.0.0.
Download the tools from dl.google.com.
Set ANDROID_HOME and ANDROID_SDK_ROOT to /opt/android-sdk.
Run npx cap sync android.
Run android/gradlew assembleDebug --no-daemon.
Copy the APK to releases/app-debug.apk.
Commit only that APK and push this branch.
Do not commit a keystore.
```

Download `releases/app-debug.apk` from the branch on GitHub. There is no file browser on the session machine. A file leaves the session by a git push, or it does not leave.

The machine has about 16 GB of RAM and 30 GB of disk. If the build is killed, use the Actions workflow. Packages installed during a session are not there in the next session. The environment cache stores what a setup script installed, and this manual keeps the SDK out of that script on purpose.

14. Failures and the reply that fixes them

Open the failed Actions step, copy from the first `FAILURE:` or `Error:` through the next 40 lines, and send:

MESSAGE — RED BUILD

```
Follow CLAUDE.md. The GitHub Android APK run failed. Fix the project and push this branch. Do not change
appId. The log is:

PASTE THE LOG HERE
```

What you see	What you send or do
Push rejected for `.github/workflows`	The GitHub token needs the `workflow` scope. From a shell: `gh auth refresh -s workflow`, then `/web-setup` again. If you connected with the GitHub App, accept the App's updated permissions on GitHub, then tell Claude to push again.
`GitHub GraphQL is not available`	Create the pull request with the **Create PR** button. Keep issue and PR commands out of the message.
No repository in the selector	A private repository appears when the Claude GitHub App is installed on the owning account and the install includes that repository. A `/web-setup` connection sees the repositories that `gh` can see. Test with `gh repo view OWNER/REPO`.
Setup script failed	The script exited non-zero, so the session did not start. This manual's Default environment has no setup script. If you added one, put `set -x` at the top, or remove the script and use section 6.
SDK 36 step failed	Paste that step's log with the red-build message. Stay on `runs-on: ubuntu-24.04`.
`Missing secret ANDROID_KEYSTORE_BASE64`	The tag ran before the four secrets existed. Add section 10, then publish the release again.
Blank screen in the installed app	`The APK opens a blank screen. Run npx cap sync android, confirm www/index.html exists, commit, and push. Do not change appId.`
App not installed, old copy present	Uninstall the old copy. A debug APK from Actions does not update a previous debug APK. A release APK updates a release APK only when the key matches and `versionCode` is higher.
Release APK will not replace the Play install	They are signed by different keys. Uninstall one before installing the other.
Play says the version is already used	Increase `versionCode`, merge, and tag a new version.
The phone shows a desktop page, zoomed out	`Add the viewport meta width=device-width, initial-scale=1 to www/index.html and cap sync.`
Fetch to your server returns a network error	The page origin is `https://localhost`. The server must allow that origin, and the URL must be `https://`. Plain `http://` is blocked.

15. What this APK can do

The installed app is a small Android program with one WebView. Your HTML, CSS, JavaScript, images, and fonts are inside the APK. With the files from section 6, the manifest permission is `android.permission.INTERNET` only.

You can have	How
Several screens	More HTML files in `www/`, linked with relative paths.
Memory on the device	`localStorage`. Removed when the app is uninstalled or when a different signature replaces it.
Images and fonts offline	Files under `www/`, relative URLs such as `images/logo.png`.

A server call	`fetch` to an `https://` URL. The caller's origin is `https://localhost`.
A basic file pick	An `input type="file"` in the HTML.
Camera, location, share sheet, local notifications, files the app stores itself	Name the official plugin in the message: `@capacitor/camera`, `@capacitor/geolocation`, `@capacitor/share`, `@capacitor/local-notifications`, `@capacitor/filesystem`. Tell Claude to `npm install` it and run `npx cap sync android`.

MESSAGE — ADD THE CAMERA

```
Follow CLAUDE.md.
Add a Take photo button using the official plugin @capacitor/camera.
Ask for the camera permission when the button is tapped.
Show the photo on the page.
Run npm install @capacitor/camera and npx cap sync android.
Commit the lockfile and any android files the plugin adds.
Do not change appId. Push a new branch.
```

The camera plugin adds permissions. The end of Claude's reply lists them. That list is part of the review in section 16.

This packaging is a poor fit for a heavy 3D game, for work that must keep running after the user leaves the app, and for an iOS build. An iOS build needs a Mac and an Apple signing certificate. It is a separate setup.

A launcher icon other than the Capacitor default is a later message: put a square PNG at `www/icon.png` and ask Claude to generate the Android icons with `@capacitor/assets` and commit the generated mipmap files. The app name under the icon is the string `app_name`, not the HTML title.

16. Before anyone else installs it

1. You installed this exact APK or Play build on your own phone and used every control.
2. The version on the screen matches `versionName` and, for a release, the git tag.
3. `appId` is the final id. Changing it later creates a different app.
4. You asked for this list and you accept it: "List every Android permission and every network host this app uses, in plain language, and point to the file for each."
5. An errand list that asks for SMS, contacts, accessibility, or device-admin is the wrong APK. Send a message to remove that permission and build again.
6. The signing key has two copies outside GitHub. `git status` does not list `upload.jks`.
7. The release you hand to someone is `app-release.apk` from the tag, or the Play listing. A debug APK from a pull request is for you.

You are responsible for what the APK does on someone else's phone. You can meet that duty without reading Java: the behavior is on your phone, the permissions are in the list, and the bytes are the artifact from the tag you published.

17. Sources

Checked 20 October 2026. Cloud-session behavior, permission modes, the GitHub proxy, installed tools, network allowlists, and resource limits follow Anthropic's Claude Code docs: `https://code.claude.com/docs/en/web-quickstart`, `https://code.claude.com/docs/en/claude-code-on-the-web`, and `https://code.claude.com/docs/en/cloud-environments`. Capacitor 8.5.3 figures were taken from a local `npx cap add android` of `@capacitor/core@8`, `@capacitor/cli@8`, and `@capacitor/android@8` on that date: app id fields, `minSdkVersion` 24, `compileSdkVersion` 36, Gradle 8.14.3, Android Gradle Plugin 8.13.0, and the default manifest permission `INTERNET`. Play App Signing behavior follows Google's Play Console help and Android's app-signing documentation. Runner image timing follows GitHub's runner-images announcement that `ubuntu-latest` moves to Ubuntu 26.04 in November 2026.